TLB and Page Table Simulator

A simulator written in C that demonstrates **virtual-to-physical address translation** using a **page table** and a **TLB** (Translation Lookaside Buffer).

Table of Contents

1. Overview
2. Features
3. Key Concepts
4. System Configuration
5. Requirements
6. Setup and Compilation
7. Execution
8. Sample Run
9. Testing
10. Project Structure
11. Team

Overview

Programs use virtual addresses, but RAM (Random Access Memory) uses physical addresses. The operating system and hardware translate between them. This project simulates that process step by step: it checks a small, fast TLB first, falls back to the page table on a miss, and handles page faults by loading pages into physical frames (evicting a page when memory is full).

Features

- Splits a 16-bit virtual address into a page number and an offset.
- TLB lookup with hit and miss detection.
- Page table walk with valid-bit checking.
- Page fault handling with page replacement when all frames are used.
- Selectable replacement policy: **LRU** (Least Recently Used) or **FIFO** (First In, First Out).
- Configurable TLB size (1 to 8 entries).
- Step-by-step explanation printed for every translation.

- Commands to display the TLB, page table and physical frames, plus statistics (hits, misses, faults, hit ratio).
- Batch mode: read addresses from a text file.

Key Concepts

Term	Meaning
Virtual address	Address used by a program
Physical address	Actual location in RAM
Page	Fixed-size block of virtual memory (4 KB here)
Frame	Fixed-size block of physical memory, same size as a page
Offset	Position inside a page; unchanged by translation
Page table	Maps each virtual page to a physical frame
TLB	Translation Lookaside Buffer, a small cache of recent page-to-frame mappings
TLB hit	Mapping found in the TLB (fastest)
TLB miss	Not in the TLB, but found in the page table
Page fault	Page not in physical memory; must be loaded

Translation: `physical address = (frame number << 12) | offset`

System Configuration

Parameter	Value
Virtual address size	16 bits (page number 4 bits + offset 12 bits)
Page size	4 KB (4096 bytes)
Virtual pages	16 (numbered 0 to 15)
Physical frames	8
TLB size	1 to 8 entries (chosen at start, default 4)
Policies	LRU or FIFO

These values are `#define` constants at the top of `tlb_simulator.c` and can be changed there.

Requirements

- A C compiler such as **GCC** (GNU Compiler Collection).
  - Windows: install MinGW, or use Code::Blocks with MinGW.
  - Linux: `sudo apt install gcc`
  - macOS: `xcode-select --install`
- A terminal or command prompt.

Setup and Compilation

```
git clone <your-repository-url>
cd <repository-folder>
gcc -Wall -o tlb_simulator tlb_simulator.c
```

The program compiles without warnings.

Execution

Interactive mode

```
./tlb_simulator          # Windows: tlb_simulator.exe
```

You will be asked for the TLB size and the replacement policy. Then use these commands:

Input	Action
`0x2A4C` (any hex address 0x0000 to 0xFFFF)	Translate the address
`r`	Translate a random address
`t`	Show the TLB, page table and physical frames
`s`	Show statistics
`c`	Change TLB size/policy and reset
`q`	Quit

Batch mode

```
./tlb_simulator sample_addresses.txt
```

Reads one hex address per line, translates each (TLB size 4, LRU), and prints statistics at the end.

Sample Run

Input file `sample_addresses.txt`:

```
0x2A4C
0x2A4C
0x2B00
0x1000
0x3000
0x4000
0x5000
0x1000
```

Expected results (TLB size 4, LRU):

#	Virtual	Page	Result	Physical
1	0x2A4C	2	Page fault	0x0A4C
2	0x2A4C	2	TLB hit	0x0A4C
3	0x2B00	2	TLB hit	0x0B00
4	0x1000	1	Page fault	0x1000
5	0x3000	3	Page fault	0x2000
6	0x4000	4	Page fault	0x3000
7	0x5000	5	Page fault	0x4000
8	0x1000	1	TLB hit	0x1000

Statistics: 8 accesses, 3 hits, 0 misses, 5 page faults, hit ratio 37.5%.

Testing

Scenarios to verify:

1. **First access to a page** gives a page fault.
2. **Repeat access to the same page** gives a TLB hit.

3. **Different address in the same page** is a hit, with the same offset.
4. **TLB full:** access more pages than the TLB size and check that a victim is removed (the oldest used page for LRU, the first loaded for FIFO).
5. **Physical memory full:** access more than 8 different pages and check that a page is evicted and its TLB entry is removed.
6. **TLB miss with valid page:** access a page that was evicted from the TLB but is still in memory (use TLB size 1 or 2) to get a TLB miss without a page fault.
7. **Invalid input:** enter an address above 0xFFFF or non-hex text and check that an error message is shown.

Project Structure

```
.
├── tlb_simulator.c        # Source code
├── sample_addresses.txt   # Sample input for batch mode
└── README.md              # This file
```

Team

Name	Roll Number	Role
(add name)	(add roll no.)	(e.g. TLB logic)
(add name)	(add roll no.)	(e.g. page table and replacement)
(add name)	(add roll no.)	(e.g. testing and documentation)

Notes

This is an educational simulator. It runs entirely in user space as an ordinary program and does not touch real system memory or other processes.